

Mental Model and Representation

Lecture 7

Tim RMK GIGA

Departemen Teknik Informatika

Mental Model

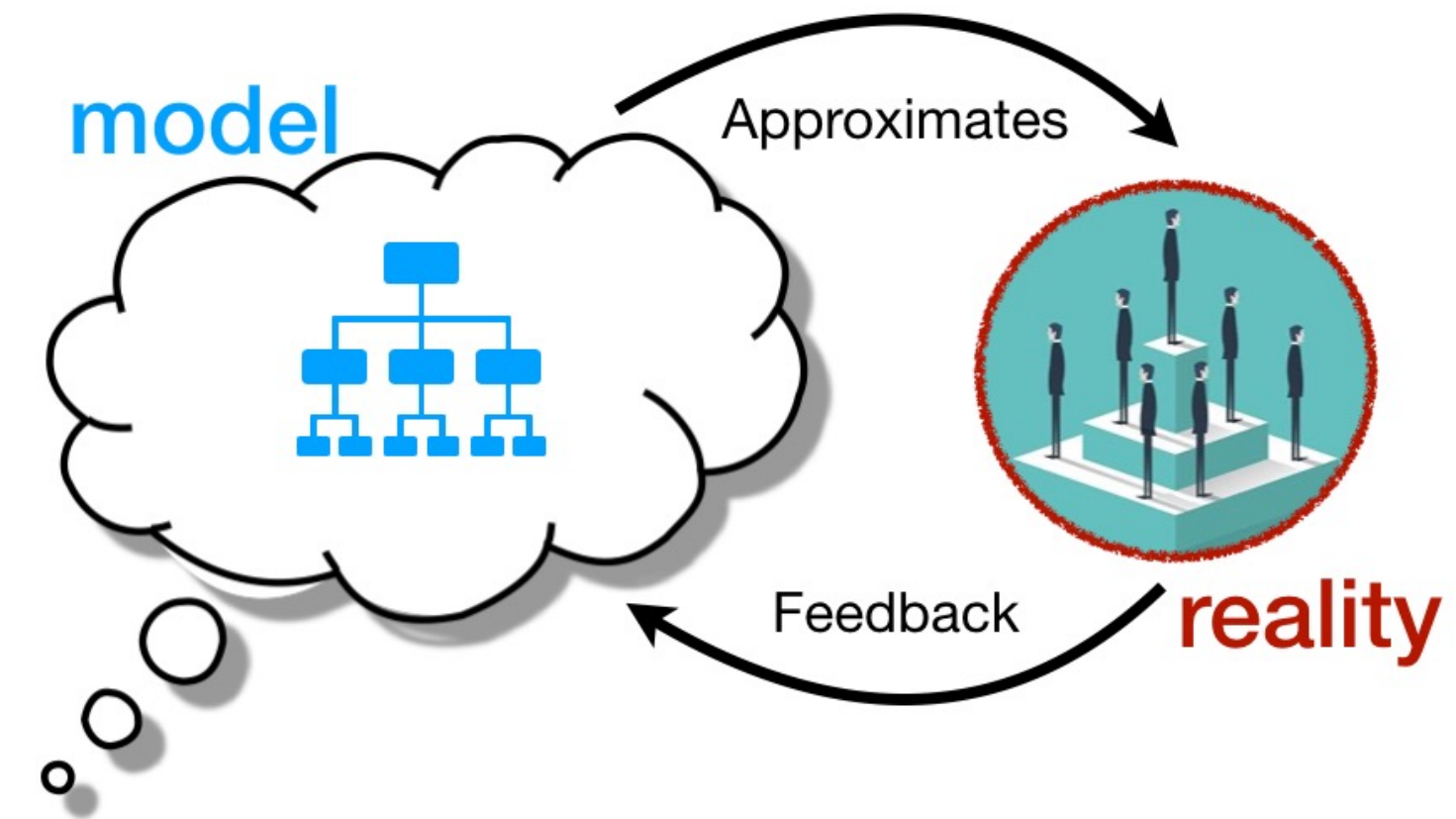
is an internal, simulatable understanding of external reality.

We use a mental model to simulate any events and use the model to predict the results of those events in a reality.

Representation

is internal symbols for external reality.

To develop a good mental model on the user's mind we need to provide a good representation of the system with which the user interacts.





The goal of today's lesson is

How to create representations that help users develop accurate mental models of our systems/apps.

Because if a representation does not match the user's mental model, it makes the user uncomfortable.



Designer = Educator



As designers of an interface, we pretty much play the role of educators.

Our goal is to **teach** our **users** how the app works **through** the **design** of the interface.

But unlike teachers, we don't generally have a chance to explain things directly to the users.

We must **design interfaces that teach users** while they are using them.

Good representations show the user exactly how the system works.

Mental Model of A Car's Air Conditioner

How do we make the air temperature colder?

How do we make the air come faster?

Do the signifiers in the car correct users' mental model?

1989 Volvo vs. 2015 Nissan Leaf

The AC controls in the Volvo car do not match our users' mental model of how the AC system works, including the color signifiers.

Nissan offers simpler AC controls, yet it still has some problems.



Matching UI to users' mental models is a valuable way to create a UI that is easily learnable by users.

5 Principles to Create Learnable Interfaces

1. Predictability
2. Synthesizability
3. Familiarity
4. Generalizability
5. Consistency



1. Predictability

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Quisque molestie nisl eu sem tristique, sit amet convallis ex
aliquam. Maecenas varius lectus hendrerit augue blandit, ut

Look at an action.

Can the user predict what will happen?

Point B

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

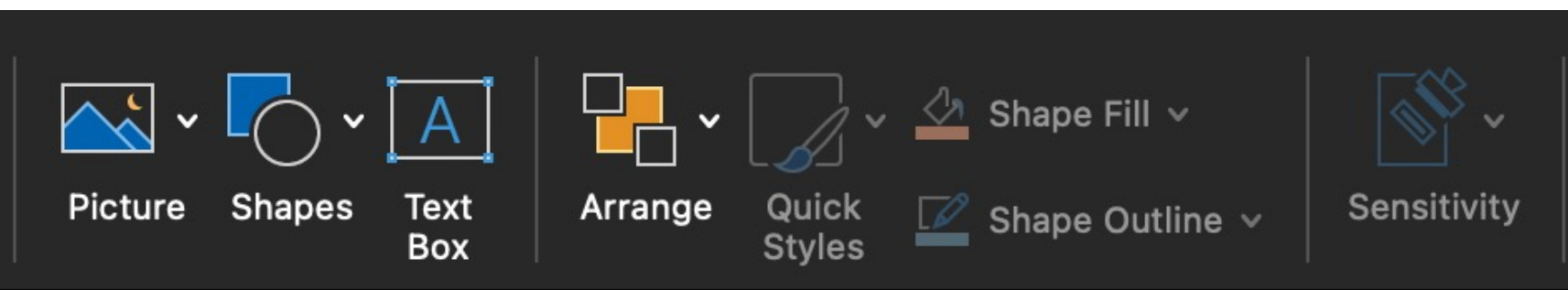
Quisque molestie nisl eu sem tristique, sit amet convallis ex
aliquam. Maecenas varius lectus hendrerit augue blandit, ut

For example, graying out the button is a good
way to help the user predict that clicking it will
do nothing.

Point C

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Quisque molestie nisl eu sem tristique, sit amet convallis ex
aliquam. Maecenas varius lectus hendrerit augue blandit, ut
posuere erat



2. Synthesizability

```
File Edit View Search Terminal Help
jimmy@ks7002:~$ python3 -V
Python 3.6.8
jimmy@ks7002:~$ sudo apt install -y python3-pip
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential cpp cpp-7 dh-python dpkg-dev
  fakeroot g++ g++-7 gcc gcc-7 gcc-7-base libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libasan4 libatomic1 libbinutils libc-dev-bin libc6-dev libcc1-0
  libcilkrts5 libdpkg-perl libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-7-dev libgomp1
  libisl19 libitm1 liblsan0 libmpc3 libmpx2 libpython3-dev libpython3.6-dev libquadmath0
  libstdc++-7-dev libtsan0 libubsan0 linux-libc-dev make manpages-dev python-pip-whl
  python3-crypto python3-dev python3-distutils python3-keyring python3-keyrings.alt
  python3-lib2to3 python3-secretstorage python3-setuptools python3-wheel python3-xdg python3.6-dev
Suggested packages:
  binutils-doc cpp-doc gcc-7-locales debian-keyring g++-multilib g++-7-multilib gcc-7-doc
  libstdc++6-7-dbg gcc-multilib autoconf automake libtool flex bison gdb gcc-doc gcc-7-multilib
  libgcc1-dbg libgomp1-dbg libitm1-dbg libatomic1-dbg libasan4-dbg liblsan0-dbg libtsan0-dbg
  libubsan0-dbg libcilkrts5-dbg libmpx2-dbg libquadmath0-dbg glibc-doc bzip libstdc++-7-doc
  make-doc python-crypto-doc gnome-keyring libkf5wallet-bin gir1.2-gnomekeyring-1.0
  python-secretstorage-doc python-setuptools-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential cpp cpp-7 dh-python dpkg-dev
  fakeroot g++ g++-7 gcc gcc-7 gcc-7-base libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libasan4 libatomic1 libbinutils libc-dev-bin libc6-dev libcc1-0
  libcilkrts5 libdpkg-perl libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-7-dev libgomp1
  libisl19 libitm1 liblsan0 libmpc3 libmpx2 libpython3-dev libpython3.6-dev libquadmath0
  libstdc++-7-dev libtsan0 libubsan0 linux-libc-dev make manpages-dev python-pip-whl
```

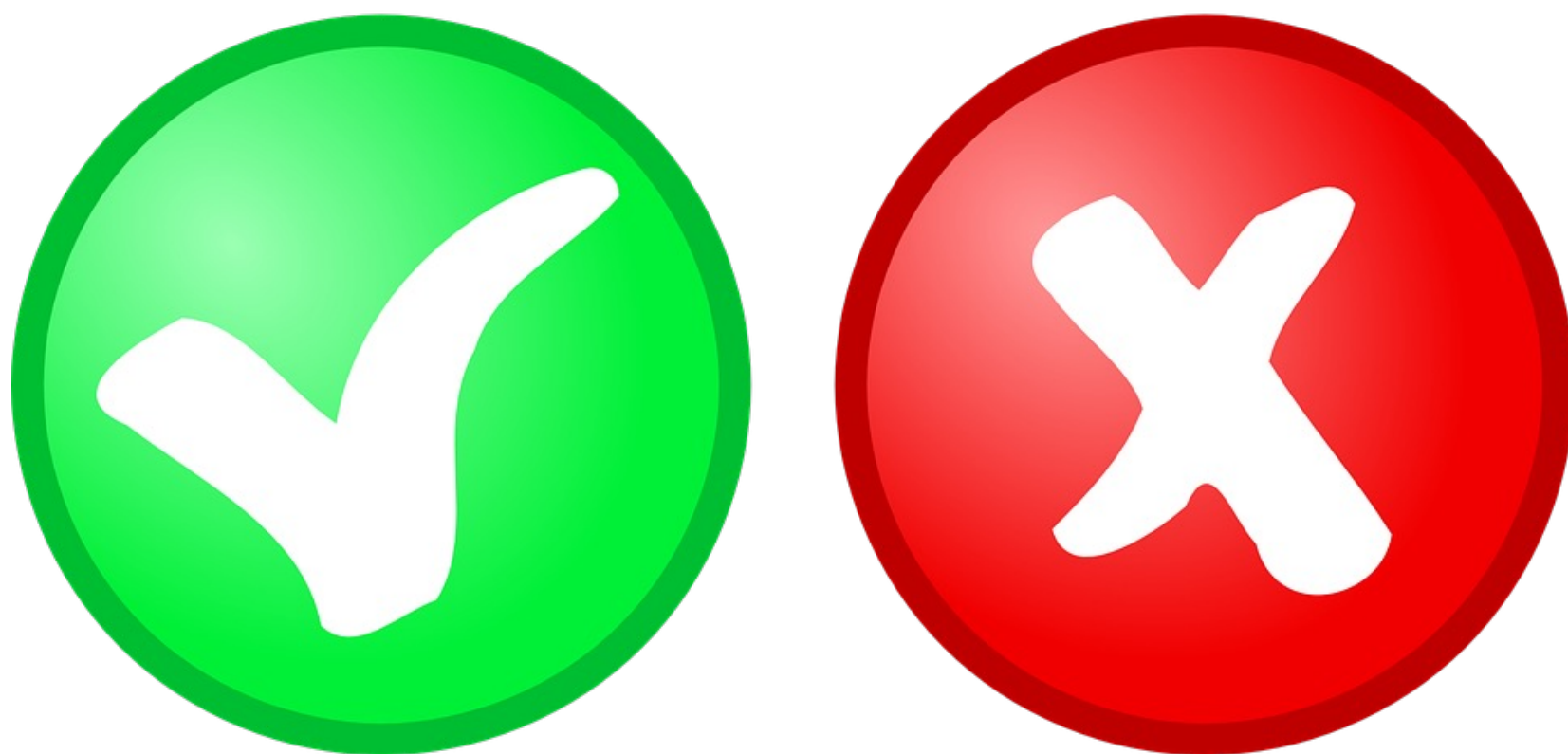
Not only should the user be able to predict the effects of an action before they perform it, but they should also be able to see the sequence of actions that led to their current state.

Point B
That cannot be easy in a GUI, but something like the log of actions they can see in the undo menu can make it easier.

Point C
Command line interfaces (CLI) are good at this. They have a log of commands that have been given in order.

This Photo by Unknown Author is licensed under [CC BY-SA-NC](#)

3. Familiarity



This is like Norman's principle of Affordances. The interfaces should leverage actions with which the user is already familiar from real-world experience.

For example, if you're trying to indicate whether something is good or bad, you might use red and green instead of yellow and blue.

4. Generalizability



Lorem ipsum dolor sit amet, consectetur adipiscing elit.
 Quisque molestie nisl eu sem tristique, sit amet convallis ex
 In Norman's principles of Consistency,
 knowledge of one user interface should be
 generalized to others.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
 Quisque molestie nisl eu sem tristique, sit amet convallis ex
 If your interface tasks are like others, like saving,
 copying, and pasting, it should perform them
 similarly.

This Photo by Unknown Author is licensed under [CC BY-SA](#)

5. Consistency



This is slightly different than Norman's principle of Consistency.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Quisque molestie nisl eu sem tristique, sit amet convallis ex

aliquam. Maecenas varius lectus hendrerit augue blandit, ut

A similar task or operation within a single interface should behave similarly.

Point B

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

For example, you wouldn't want to have Ctrl+X to cut some texts if texts are selected but close the app if no texts are selected.

Point C

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

The behavior of any action should be consistent across the interface.

posuere erat.

Learn Representation from DIY Furniture

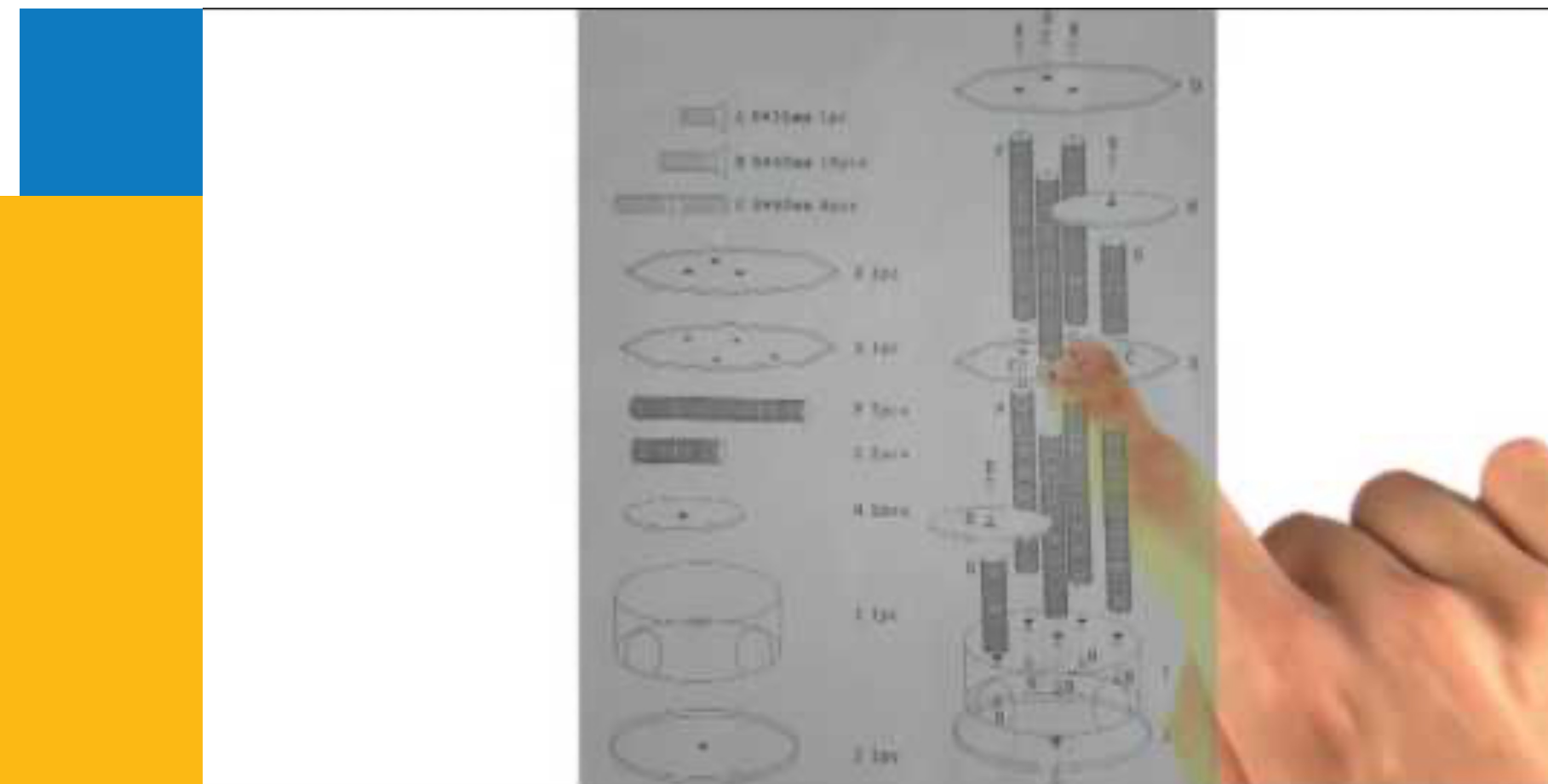
Good representations simplify complex tasks.

Users can intuitively follow instructions
and complete tasks accurately.

Mapping Visuals to Reality

Accurate representations help users
understand the physical assembly.

Visual cues like arrows enhance comprehension.



Bad Problem Representation

A hiker starts climbing a mountain at 7:00am.



Animation Overview

The hiker goes up the hill on one day, stays overnight, then descends the hill the next day.

Key Question

Was the hiker ever at the same point at the same time on both days?

What do you think? 🤔

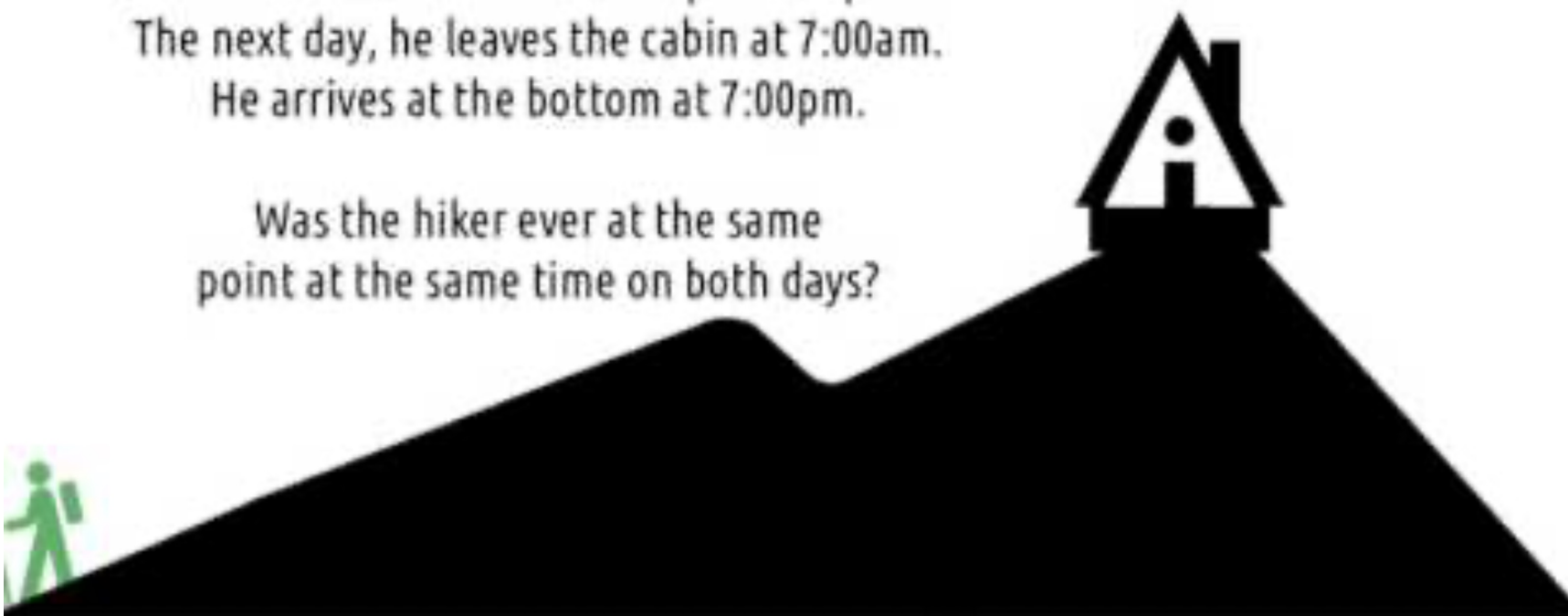
Poin C

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque molestie nisl eu sem tristique, sit amet convallis ex aliquam. Maecenas varius lectus hendrerit augue blandit, ut posuere erat.

Good Problem Representation

A hiker starts climbing a mountain at 7:00am.
He arrives at the cabin on top at 7:00pm.
The next day, he leaves the cabin at 7:00am.
He arrives at the bottom at 7:00pm.

Was the hiker ever at the same point at the same time on both days?



Animation Overview

The hiker goes up the hill on one day, stays overnight, then descends the hill the next day.

Key Question

Was the hiker ever at the same point at the same time on both days? The answer is YES.

Tweaking the Representation

Visualize both days simultaneously.

Easier to see that the hiker must pass himself.

Good representation transforms a complex problem into an easy one.

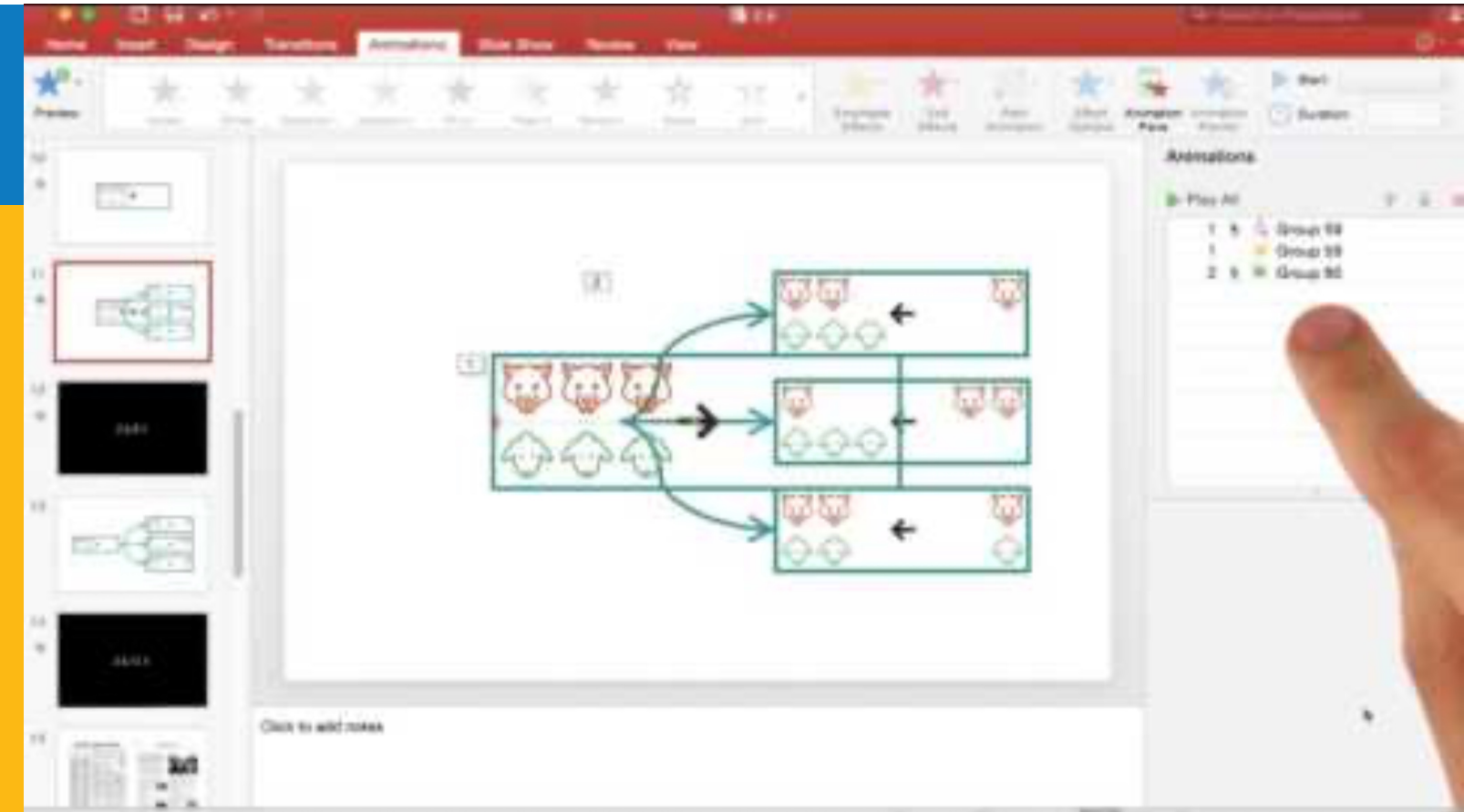
Representation in UI

Good representations in user interfaces align users' mental model with the real state of the system.

Example of Good Representations in UI

Scroll bars represent the relative position in a document.

Highlight markers indicate the current selection.





While representations help users form accurate mental models of a system,
Metaphors & Analogies further enhance understanding by relating new interfaces to familiar concepts.

Wall Street Journal

The digital version mirrors the print edition.

A familiar layout helps transition to the online version.

Slack

Described as a chat app for organizations.

Leverages the familiar concept of chat applications.

Design Principles Revisited



Reasoning by Analogy and Metaphor

Users reason by analogy to past interfaces or metaphors to the real world.

Principle of consistency is crucial to align with these analogies and metaphors



Teaching Through Interface Design

Interfaces should teach users how the system works.

Echoes (perceived) affordances: system appearance should indicate usage.

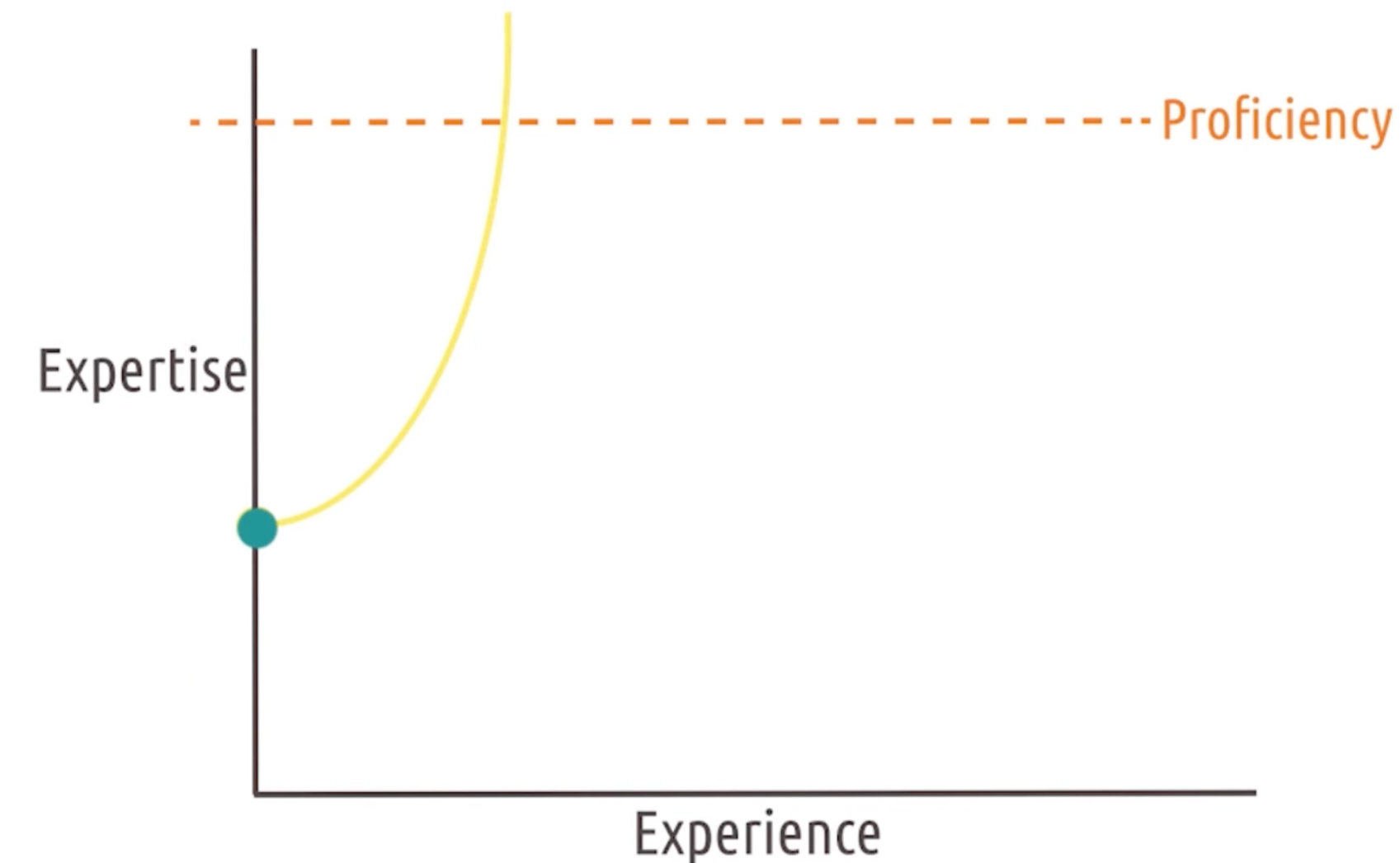


Importance of Representations

Representations map the interface to the task at hand.

Users can foresee the mapping between their actions in the interface and real-world results.

Learning Curve



Achieving Rapid **Mastery**

- Starting with Initial Expertise
Leverage existing user knowledge.
Example: Familiar icons and UI elements.
- Rapid Expertise Growth
Use clear representations / intuitive design.
Example: Visual cues guide user interaction.

Point C

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Quisque molestie nisl eu sem tristique, sit amet convallis ex aliquam. Maecenas varius lectus hendrerit augue blandit, ut posuere erat.



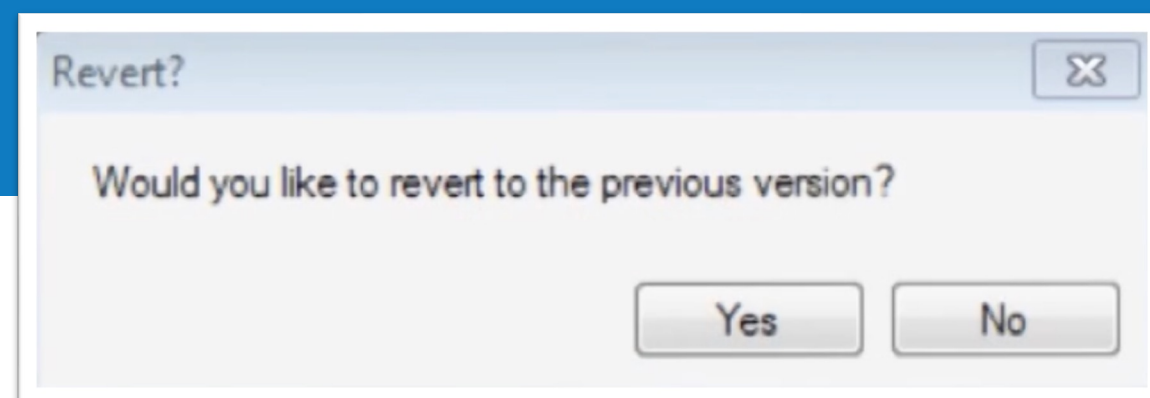
User Errors



Mistakes

Occur when the user has the wrong mental model, thus performs the wrong action

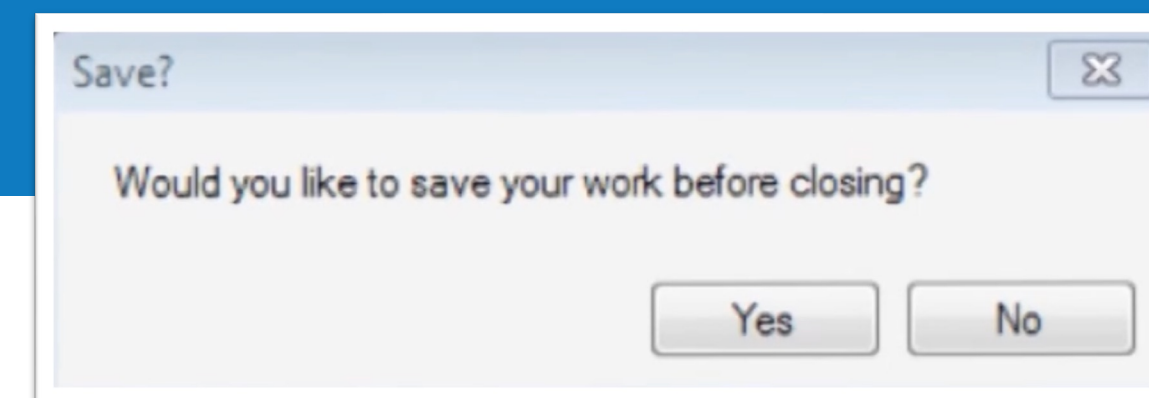
Example: Misunderstanding a prompt like "revert to the original file" due to unfamiliar terminology



Slips

Occur when the user has the right mental model but performs the wrong action anyway

Example: Clicking "No" instead of "Yes" due to button placement or default selection





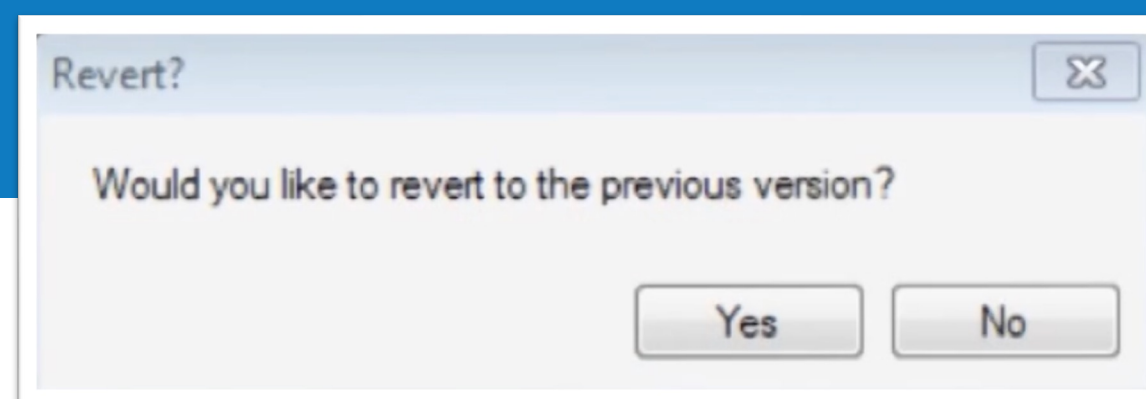
User Errors



to Prevent Mistakes

Use clear and
familiar terminology

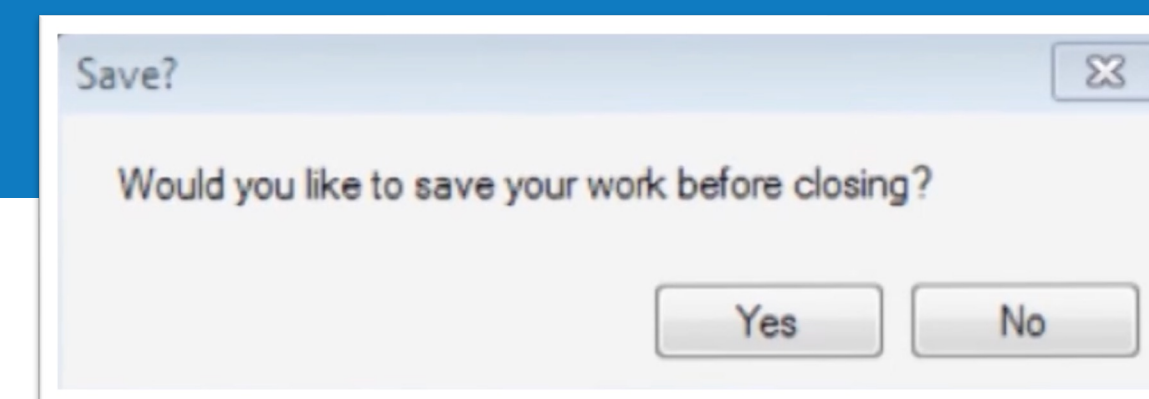
Provide options that align with user mental models



to Prevent Slips

Ensure intuitive button placement
and default selections

Align interface actions with user expectations



Mistakes

Don Norman's Three Categories of Mistakes

1. Rule-Based Mistakes

User correctly assesses the state of the world
but makes the wrong decision.

Example: Clicking "No" thinking it will save changes.

2. Knowledge-Based Mistakes

User incorrectly assesses the state of the world.

Example: Not realizing changes were made and
choosing not to save.

3. Memory Lapse Mistakes

User forgets to fully execute a plan.

Example: Shutting down the computer without
addressing a save prompt.

Slips

Don Norman's Two Categories of Slips

1. Action-Based Slips

User performs the wrong action or the right action on the wrong object.

Example:

Clicking the wrong button or right-clicking instead of left-clicking.



2. Memory Lapse Slips

User forgets to perform an action they knew to do.

Example:

Forgetting to start a timer on a microwave

Example (and prevention):

Dialog boxes prompt users to save work before closing to prevent memory lapse slips.

**Do you want to keep this new document
"Untitled"?**

You can choose to save your changes, or delete this document immediately. You can't undo this action.

Save As:

Tags:

Where:  

File Format: 

Delete

Cancel

Save

Preventing User Errors

Routine Errors

Use consistent design practices.

Example: Standard dialog layouts.

Memory Lapse Errors

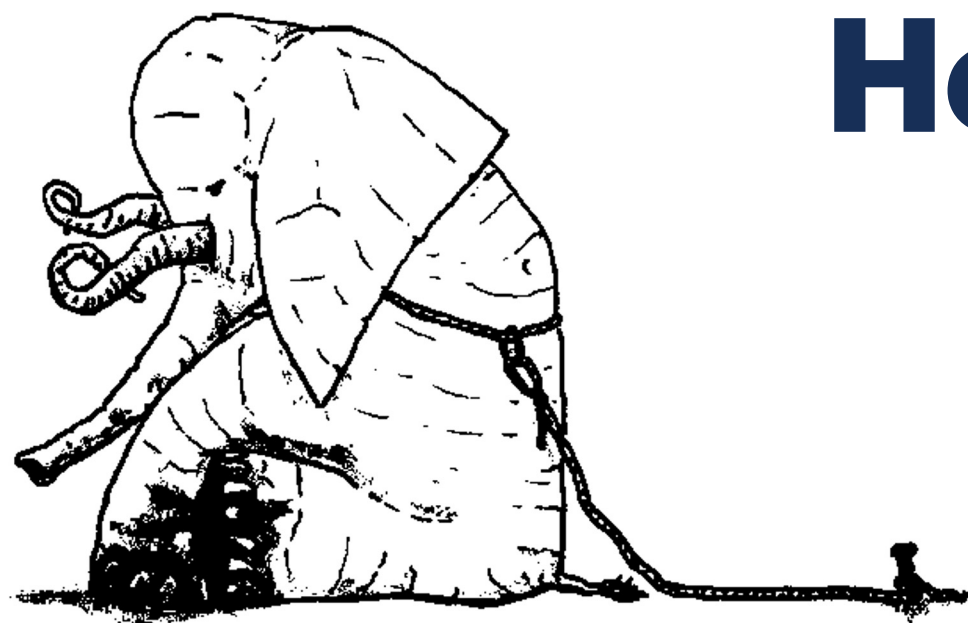
Offload memory demands from the user to the computer.

Example: Automatic reminders or prompts.

Rule-Based and Knowledge-Based Errors

Use good representations to help users develop accurate mental models.

Learned Helplessness



This Photo by Unknown Author is licensed under [CC BY-NC-ND](#)

Users feel helpless to use the system.

No mapping between input and output leads to frustration.

Breaks in the Feedback Cycle

No discernible connection between input and output.

User repeatedly acts without helpful output.

Importance of Effective Feedback

Correct actions lead to goal accomplishment.

Errors provide feedback on how to avoid next time.

Ensure clear and helpful feedback from the interface.

Maintain a strong connection between user actions and system responses.

Expert Blind Spot

Professional basketball player knowing hand placement without thinking.

Designers and operators performing tasks automatically.

Key Principle: "I Am Not My User"

Designers teach users through the interface design.

Do not assume users are experts.

Ensure interfaces are intuitive for all users.





Thank You

Tim RMK GIGA

